

ESP32 WiFi Provisioning Test Suite

Comprehensive multi-scheme Wi-Fi provisioning and diagnostic playground for ESP32 (Arduino framework) using:

- Native Arduino WiFiProv BLE provisioning (base example)
- SoftAP provisioning with enhanced credential extraction + connection state machine
- External HTML/web-based provisioning library ([WiFiProv-ESP32](#)) in simple, configuration, and advanced forms
- Hardware/spec introspection utility for flash, partitions, MAC addresses, and memory statistics

The repository is organized as a set of independent PlatformIO build environments, each focused on a single demonstration scenario. This allows rapid switching, isolated binary sizes, and iterative experimentation without cross-contamination of logic.

Repository Structure

```
platformio.ini          # Environment definitions & global build settings
include/                # Shared headers (if any future additions)
lib/WiFiProv-ESP32/     # External provisioning web server library (HTML +
logic)                  #
src/
  WiFiProv/             # Base BLE provisioning example (simple BLE QR
flow)                   #
  WiFi_Provisioning.cpp
  WiFiProv_SoftAP/      # SoftAP provisioning + advanced post-provision
connect + diagnostics
  WiFi_Provisioning_SoAP.cpp
  WiFiProv-extLib-simple/ # External library simple example (minimal UI)
simple.cpp
  WiFiProv-extLib-configuration/ # External library configuration example
(custom theming + input validation)
  configuration.cpp
  WiFiProv-extLib-advanced/ # External library advanced example (preferences
persistence + API key management)
  advanced.cpp
  ESP32_Specs_Reader/    # Hardware/spec diagnostics tool
ESP32_Specs_Reader.cpp
```

Build Environments (PlatformIO)

Environment	Purpose	Transport	Extras	Approx Flash	Approx RAM
-------------	---------	-----------	--------	--------------	------------

Environment	Purpose	Transport	Extras	Approx Flash	Approx RAM
WiFiProv	Base BLE provisioning	BLE	QR POP	~1.58 MB (50%)	~56 KB (17%)
WiFiProv_SoftAP	SoftAP provisioning + state machine + diagnostics	SoftAP	DHCP resilience, BSSID scan	~1.60 MB (51%)	~58 KB (18%)
WiFiProv_extLib_simple	External lib minimal web UI	SoftAP (internal captive portal)	Hidden extra fields	~0.84 MB (27%)	~45 KB (14%)
WiFiProv_extLib_configuration	External lib custom theming + input validation	SoftAP captive portal	Input code check	~0.84 MB (27%)	~45 KB (14%)
WiFiProv_extLib_advanced	External lib advanced persistent API key, button trigger	SoftAP captive portal	Preferences storage & conditional UI	~0.85 MB (27%)	~45 KB (14%)
Specs	Device spec & partition introspection	N/A	Reporting only	~0.28 MB (9%)	~21 KB (7%)

Flash/RAM figures taken from recent successful builds (release, -03). Values vary slightly with future edits.

Core Example: WiFiProv/WiFi_Provisioning.cpp (Base BLE Flow)

The base example demonstrates the minimal BLE provisioning flow provided by Arduino’s WiFiProv component:

1. Initialize Serial + NVS.
2. Register a provisioning event handler (SysProvEvent).
3. Configure station mode & clear prior sessions (optional).
4. Start BLE provisioning using POP security (Security Level 1) and service name.
5. Print a QR code embedding scheme, service name, and POP so the mobile app can auto-fill parameters.
6. React to events (credentials received, success/fail, end). Logic purposefully ends there—post-connect behavior can be layered in.

Why it’s foundational: All other provisioning variants extend the idea of obtaining credentials securely; the BLE base proves the minimal contract between device and provisioning client.

SoftAP Advanced Provisioning: `WiFiProv_SoftAP/WiFi_Provisioning_SoAP.cpp`

Highlights:

- Starts SoftAP-based provisioning (for situations where BLE is unreliable or disabled).
- Extracts credentials via `esp_wifi_get_config()` (since Arduino's WiFi API does not expose password directly post-provision).
- Implements a robust connection state machine (phases: IDLE → WAIT_DELAY → ASSOC → DHCP → STABLE) to mitigate mesh network steering and slow DHCP responses.
- Adds BSSID scanning + optional locking to strongest AP instance to reduce roaming flakiness.
- Periodic diagnostics (`RSSI`, IP, latency pseudo-ping, heap, uptime, current phase) every 5 seconds.
- Mesh / enterprise etc. tuning macros (override via `build_flags`):
 - `WIFI_CONNECT_DELAY_MS`, `WIFI_ASSOC_TIMEOUT_MS`, `WIFI_DHCP_TIMEOUT_MS_EXT`,
`WIFI_RETRY_BACKOFF_MS`, `WIFI_MAX_ASSOC_RETRIES`, `WIFI_SCAN_ON_FAIL`,
`WIFI_LOCK_BEST_BSSID`.

This environment is ideal for validating network edge cases: delayed DHCP, AP steering, and credential persistence.

External Library Provisioning Examples (`lib/WiFiProv-ESP32`)

All three examples use an embedded captive portal + HTML/JS interface served via SoftAP. They leverage callbacks for validation, persisting configuration, and conditional UI adjustments.

1. Simple (`WiFiProv-extLib-simple/simple.cpp`)

- Minimal onboarding.
- Hides optional input and reset fields.
- On success prints SSID & password.
- Best for quick smoke tests of the external library integration.

2. Configuration (`WiFiProv-extLib-configuration/configuration.cpp`)

- Demonstrates full customization of branding/theme (color, logo SVG), text sections, and an input field validation routine (`onInputCheck`).
- Factory reset callback included.
- Shows how to adapt UI without touching library internals.

3. Advanced (`WiFiProv-extLib-advanced/advanced.cpp`)

- Adds persistent storage using `Preferences` for SSID/password/API key.
 - Button-triggered re-provision (GPIO 9). Debounces by simple state read.
 - Dynamically hides input field if API key already saved (runtime config mutation).
 - Includes factory reset clearing all keys and values.
 - Illustrates multi-stage UX + conditional configuration adaptation.
-

Hardware / System Introspection: `ESP32_Specs_Reader/ESP32_Specs_Reader.cpp`

Provides a quick diagnostic snapshot:

- Chip model, feature flags (BT/BLE, core count), revision.
- Detected flash size and basic configuration pointers.
- PSRAM presence & free heap metrics (including lifetime minimum).
- MAC addresses for STA, SoftAP, optionally BT/Ethernet.
- Enumerates partition table (APP and DATA partitions) for sizing guidance.

Use this environment before selecting partition schemes or when validating unknown boards with mismatched flash claims.

Usage Workflow (General)

1. Select environment (e.g. `WiFiProv_SoftAP`) in VS Code / run via PlatformIO CLI: `platformio run -e WiFiProv_SoftAP --target upload`.
2. Open Serial Monitor at configured baud (default `115200`).
3. For provisioning examples:
 - BLE: Scan QR code with official provisioning app; follow prompts.
 - SoftAP: Connect to advertised AP; supply target network credentials.
 - External lib: Join AP and browse to captive portal page (often `192.168.4.1`).
4. Observe console logs for event phases, credential acceptance, connection status.
5. (SoftAP advanced) Watch diagnostic prints until IP lease acquired and phase transitions to STABLE.
6. Extend or integrate post-connect logic (MQTT, HTTP, cloud sync) in the respective loop or success callbacks.